```python
"""
AQI Guardian — Air Quality Prediction Dashboard
Streamlit Deployment | Logistic Regression Model
"""

import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import warnings
warnings.filterwarnings("ignore")

# ── Page Config ──────────────────────────────────────────────
st.set_page_config(
    page_title="AQI Guardian",
    page_icon="🌿",
    layout="wide",
    initial_sidebar_state="expanded"
)

# ── Custom CSS ───────────────────────────────────────────────
st.markdown("""
<style>
    @import
url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600;700&display=sw
ap');

    html, body, [class*="css"] { font-family: 'Inter', sans-serif; }

    .main { background: #0f1117; }

    .hero-banner {
        background: linear-gradient(135deg, #1a1a2e 0%, #16213e 50%, #0f3460 100%);
        border-radius: 20px;
        padding: 40px 48px;
        margin-bottom: 28px;
        border: 1px solid rgba(99,179,237,0.2);
    }
    .hero-title {
        font-size: 2.8rem;
        font-weight: 700;
        color: #ffffff;
        margin: 0;
```

```
        letter-spacing: -0.5px;
    }
    .hero-subtitle {
        font-size: 1.1rem;
        color: #90cdf4;
        margin-top: 8px;
    }
    .hero-badge {
        display: inline-block;
        background: rgba(99,179,237,0.15);
        border: 1px solid rgba(99,179,237,0.4);
        border-radius: 20px;
        padding: 4px 16px;
        font-size: 0.8rem;
        color: #90cdf4;
        margin-top: 12px;
    }

    .metric-card {
        background: linear-gradient(145deg, #1e2a3a, #162032);
        border: 1px solid rgba(99,179,237,0.15);
        border-radius: 16px;
        padding: 20px 24px;
        text-align: center;
        transition: transform 0.2s;
    }
    .metric-card:hover { transform: translateY(-3px); }
    .metric-label {
        font-size: 0.75rem;
        color: #718096;
        text-transform: uppercase;
        letter-spacing: 1px;
        margin-bottom: 6px;
    }
    .metric-value {
        font-size: 2rem;
        font-weight: 700;
        color: #ffffff;
    }
    .metric-delta {
        font-size: 0.8rem;
        color: #68d391;
        margin-top: 4px;
    }
```

```
.aqi-result-card {
    border-radius: 20px;
    padding: 32px;
    text-align: center;
    margin: 16px 0;
    border: 2px solid;
}
.section-header {
    font-size: 1.3rem;
    font-weight: 600;
    color: #e2e8f0;
    padding-bottom: 8px;
    border-bottom: 2px solid rgba(99,179,237,0.2);
    margin-bottom: 20px;
}

.info-pill {
    display: inline-block;
    background: rgba(104,211,145,0.1);
    border: 1px solid rgba(104,211,145,0.3);
    border-radius: 12px;
    padding: 4px 12px;
    font-size: 0.75rem;
    color: #68d391;
    margin: 3px;
}

div[data-testid="stSidebar"] {
    background: #111827 !important;
    border-right: 1px solid rgba(99,179,237,0.1);
}
.sidebar-section {
    background: rgba(99,179,237,0.05);
    border-radius: 12px;
    padding: 16px;
    margin: 12px 0;
    border: 1px solid rgba(99,179,237,0.1);
}

.stButton button {
    background: linear-gradient(135deg, #3182ce, #2b6cb0);
    color: white;
    border: none;
    border-radius: 12px;
    padding: 14px 28px;
```

```
        font-size: 1rem;
        font-weight: 600;
        width: 100%;
        transition: all 0.2s;
        cursor: pointer;
    }
    .stButton button:hover {
        background: linear-gradient(135deg, #2b6cb0, #2c5282);
        transform: translateY(-1px);
    }

    .stSlider .stSlider { color: #3182ce; }
    .stNumberInput input { background: #1a2535; color: white; border-color:
rgba(99,179,237,0.2); }

    .realtime-dot {
        display: inline-block;
        width: 8px; height: 8px;
        background: #68d391;
        border-radius: 50%;
        animation: pulse 1.5s infinite;
        margin-right: 6px;
    }
    @keyframes pulse {
        0%,100% { opacity:1; } 50% { opacity:0.3; }
    }

    .tab-content { padding: 8px 0; }
    .warning-box {
        background: rgba(245,101,101,0.1);
        border: 1px solid rgba(245,101,101,0.3);
        border-radius: 12px;
        padding: 16px;
        margin: 12px 0;
    }
    .success-box {
        background: rgba(104,211,145,0.1);
        border: 1px solid rgba(104,211,145,0.3);
        border-radius: 12px;
        padding: 16px;
        margin: 12px 0;
    }
    footer {visibility: hidden;}
    #MainMenu {visibility: hidden;}
</style>
```

```
""", unsafe_allow_html=True)


# ── AQI Utilities ──────────────────────────────────────────────

AQI_CATEGORIES = {
    'Good':         {'range': (0,50),   'color': '#38a169', 'emoji': '😊', 'bg':
'rgba(56,161,105,0.1)',  'border': '#38a169'},
    'Satisfactory': {'range': (51,100), 'color': '#68d391', 'emoji': '🙂', 'bg':
'rgba(104,211,145,0.1)', 'border': '#68d391'},
    'Moderate':     {'range': (101,200),'color': '#ecc94b', 'emoji': '😐', 'bg':
'rgba(236,201,75,0.1)',  'border': '#ecc94b'},
    'Poor':         {'range': (201,300),'color': '#ed8936', 'emoji': '😷', 'bg':
'rgba(237,137,54,0.1)',  'border': '#ed8936'},
    'Very Poor':    {'range': (301,400),'color': '#f56565', 'emoji': '🤢', 'bg':
'rgba(245,101,101,0.1)', 'border': '#f56565'},
    'Severe':       {'range': (401,500),'color': '#9f7aea', 'emoji': '☠', 'bg':
'rgba(159,122,234,0.1)', 'border': '#9f7aea'},
}

HEALTH_ADVICE = {
    'Good':         'Air quality is excellent. Enjoy outdoor activities freely.',
    'Satisfactory': 'Air quality is acceptable. Unusually sensitive people should
reduce prolonged exertion.',
    'Moderate':     'Sensitive groups may experience health effects. Reduce
extended outdoor exertion.',
    'Poor':         'Everyone may experience health effects. Sensitive groups at
higher risk. Wear N95 mask.',
    'Very Poor':    'Health alert! Everyone should avoid prolonged outdoor
exertion. Stay indoors.',
    'Severe':       'Health emergency! Avoid all outdoor activity. Keep windows
shut. Seek medical attention if unwell.',
}

def compute_aqi(so2, no2, rspm, spm, pm2_5):
    """Simplified AQI computation"""
    aqi = 0.30*pm2_5 + 0.25*rspm + 0.20*no2 + 0.15*spm + 0.10*so2
    return round(aqi, 1)

def categorize_aqi(aqi_val):
    for cat, info in AQI_CATEGORIES.items():
        lo, hi = info['range']
        if lo <= aqi_val <= hi:
            return cat
    return 'Severe'
```

```
def get_aqi_color(aqi_val):
    return AQI_CATEGORIES[categorize_aqi(aqi_val)]['color']

def simulate_model_predict(so2, no2, rspm, spm, pm2_5, temperature, humidity, city,
season, month, year):
    """Simulates logistic regression prediction (replace with loaded joblib
model)"""
    aqi_val = compute_aqi(so2, no2, rspm, spm, pm2_5)
    category = categorize_aqi(aqi_val)
    # Simulate probabilities
    cats = list(AQI_CATEGORIES.keys())
    idx = cats.index(category)
    probs = np.random.dirichlet(np.array([0.1]*6))
    probs[idx] = np.random.uniform(0.55, 0.85)
    probs = probs / probs.sum()
    return category, aqi_val, dict(zip(cats, probs))


# ── Sidebar ──────────────────────────────────────────────────
with st.sidebar:
    st.markdown("""
    <div style='text-align:center;padding:16px 0 8px'>
        <span style='font-size:2rem'>🌿</span>
        <h2 style='color:#90cdf4;margin:4px 0;font-size:1.2rem'>AQI Guardian</h2>
        <p style='color:#718096;font-size:0.75rem;margin:0'>AI-Powered Air Quality
Prediction</p>
    </div>
    """, unsafe_allow_html=True)

    st.markdown("---")
    st.markdown("#### 📍 Location Info")

    city = st.selectbox("City",
['Delhi','Mumbai','Chennai','Kolkata','Bengaluru','Hyderabad','Ahmedabad','Pune'])
    season = st.selectbox("Season", ['Winter','Spring','Summer','Monsoon'])
    month = st.slider("Month", 1, 12, 6)
    year = st.selectbox("Year", [2022,2023,2024,2025])

    st.markdown("---")
    st.markdown("#### 🧪 Pollutant Levels (µg/m³)")

    so2   = st.slider("SO₂",   0.0, 100.0, 12.0, 0.5)
    no2   = st.slider("NO₂",   0.0, 150.0, 38.0, 0.5)
    rspm  = st.slider("RSPM",  0.0, 300.0, 85.0, 1.0)
```

```
    spm   = st.slider("SPM",   0.0, 500.0,140.0, 1.0)
    pm2_5 = st.slider("PM2.5", 0.0, 250.0, 55.0, 0.5)

    st.markdown("---")
    st.markdown("#### 🌡 Weather Conditions")

    temperature = st.slider("Temperature (°C)", -5.0, 50.0, 28.0, 0.5)
    humidity    = st.slider("Humidity (%)",       10.0,100.0, 65.0, 1.0)

    st.markdown("---")
    predict_btn = st.button("🔍 Predict AQI Category")

    st.markdown("""
    <div style='margin-
top:16px;padding:12px;background:rgba(99,179,237,0.05);border-
radius:10px;border:1px solid rgba(99,179,237,0.1)'>
        <p style='color:#718096;font-size:0.7rem;margin:0;line-height:1.6'>
            <b style='color:#90cdf4'>Model:</b> Logistic Regression
(Multinomial)<br>
            <b style='color:#90cdf4'>Dataset:</b> India Air Quality (Kaggle)<br>
            <b style='color:#90cdf4'>Classes:</b> 6 AQI Categories (CPCB)
        </p>
    </div>
    """, unsafe_allow_html=True)


# ── Hero Banner ──────────────────────────────────────────────
st.markdown("""
<div class='hero-banner'>
    <div style='display:flex;align-items:center;gap:16px'>
        <span style='font-size:3.5rem'>🌿</span>
        <div>
            <p class='hero-title'>AQI Guardian</p>
            <p class='hero-subtitle'>Real-Time Air Quality Prediction powered by
Machine Learning</p>
            <span class='hero-badge'><span class='realtime-dot'></span>Logistic
Regression · CPCB Standard · 6 Categories</span>
        </div>
    </div>
</div>
""", unsafe_allow_html=True)


# ── Top Metric Strip ─────────────────────────────────────────
aqi_live = compute_aqi(so2, no2, rspm, spm, pm2_5)
```

```
cat_live  = categorize_aqi(aqi_live)
cat_info  = AQI_CATEGORIES[cat_live]

c1,c2,c3,c4,c5 = st.columns(5)
metrics = [
    ("Live AQI",     f"{aqi_live:.0f}",    cat_info['color'],  "Computed from
inputs"),
    ("Category",     cat_live,              cat_info['color'],  cat_info['emoji']),
    ("PM2.5",        f"{pm2_5:.1f} µg/m³", "#90cdf4",          "Fine particles"),
    ("NO₂",          f"{no2:.1f} µg/m³",   "#fbd38d",          "Nitrogen dioxide"),
    ("Humidity",     f"{humidity:.0f}%",    "#68d391",          "Relative
humidity"),
]
for col, (label, val, color, delta) in zip([c1,c2,c3,c4,c5], metrics):
    with col:
        st.markdown(f"""
        <div class='metric-card'>
            <p class='metric-label'>{label}</p>
            <p class='metric-value' style='color:{color}'>{val}</p>
            <p class='metric-delta'>{delta}</p>
        </div>
        """, unsafe_allow_html=True)

st.markdown("<br>", unsafe_allow_html=True)


# ── Tabs ──────────────────────────────────────────────────────────
tab1, tab2, tab3, tab4 = st.tabs(["🎯 Prediction", "📊 Analytics", "🗺️ AQI Map",
"ℹ️ About"])


# ══════════════════════════════════════════════════════════════════════
#  TAB 1 — PREDICTION
# ══════════════════════════════════════════════════════════════════════
with tab1:
    if predict_btn:
        with st.spinner("🧠 Running Logistic Regression model..."):
            import time; time.sleep(0.6)
            category, aqi_val, probs = simulate_model_predict(
                so2, no2, rspm, spm, pm2_5, temperature, humidity,
                city, season, month, year
            )
            cinfo = AQI_CATEGORIES[category]

        # ── Result Card ──
```

```
        st.markdown(f"""
        <div class='aqi-result-card' style='background:{cinfo["bg"]};border-
color:{cinfo["border"]}'>
            <p style='font-size:4rem;margin:0'>{cinfo["emoji"]}</p>
            <p style='font-size:2.5rem;font-
weight:700;color:{cinfo["color"]};margin:8px 0'>{category}</p>
            <p style='font-size:1.1rem;color:#a0aec0'>AQI Value: <b
style='color:#ffffff'>{aqi_val:.0f}</b></p>
            <p style='font-size:0.95rem;color:#e2e8f0;max-width:480px;margin:12px
auto 0'>{HEALTH_ADVICE[category]}</p>
        </div>
        """, unsafe_allow_html=True)

        col_a, col_b = st.columns([3, 2])

        with col_a:
            # ── Probability Gauge Chart ──
            st.markdown("<p class='section-header'>📊 Class Probabilities</p>",
unsafe_allow_html=True)
            cats = list(probs.keys())
            prob_vals = [round(v*100, 1) for v in probs.values()]
            colors_bar = [AQI_CATEGORIES[c]['color'] for c in cats]

            fig_prob = go.Figure(go.Bar(
                x=cats, y=prob_vals,
                marker_color=colors_bar,
                text=[f"{v:.1f}%" for v in prob_vals],
                textposition='outside',
                hovertemplate="<b>%{x}</b><br>Probability:
%{y:.1f}%<extra></extra>"
            ))
            fig_prob.update_layout(
                paper_bgcolor='rgba(0,0,0,0)',
                plot_bgcolor='rgba(0,0,0,0)',
                font_color='#e2e8f0',
                height=320,
                margin=dict(l=0,r=0,t=20,b=0),
                yaxis=dict(showgrid=True, gridcolor='rgba(255,255,255,0.05)',
title='Probability (%)', range=[0,105]),
                xaxis=dict(showgrid=False)
            )
            st.plotly_chart(fig_prob, use_container_width=True)

        with col_b:
            # ── AQI Gauge ──
```

```
            st.markdown("<p class='section-header'>🎯 AQI Gauge</p>",
unsafe_allow_html=True)
            fig_gauge = go.Figure(go.Indicator(
                mode="gauge+number+delta",
                value=aqi_val,
                delta={'reference': 100, 'increasing': {'color': '#f56565'},
'decreasing': {'color': '#68d391'}},
                title={'text': "AQI Value", 'font': {'color': '#e2e8f0'}},
                number={'font': {'color': cinfo['color'], 'size': 48}},
                gauge={
                    'axis': {'range': [0, 500], 'tickcolor': '#718096'},
                    'bar': {'color': cinfo['color']},
                    'bgcolor': 'rgba(0,0,0,0)',
                    'steps': [
                        {'range': [0, 50],   'color': 'rgba(56,161,105,0.15)'},
                        {'range': [51, 100], 'color': 'rgba(104,211,145,0.12)'},
                        {'range': [101, 200],'color': 'rgba(236,201,75,0.12)'},
                        {'range': [201, 300],'color': 'rgba(237,137,54,0.12)'},
                        {'range': [301, 400],'color': 'rgba(245,101,101,0.12)'},
                        {'range': [401, 500],'color': 'rgba(159,122,234,0.12)'},
                    ],
                    'threshold': {'line': {'color': 'white', 'width': 2}, 'value':
aqi_val}
                }
            ))
            fig_gauge.update_layout(
                paper_bgcolor='rgba(0,0,0,0)',
                font={'color': '#e2e8f0'},
                height=280,
                margin=dict(l=20,r=20,t=20,b=0)
            )
            st.plotly_chart(fig_gauge, use_container_width=True)

            # Health info
            st.markdown(f"""
            <div style='background:rgba(99,179,237,0.05);border-
radius:12px;padding:14px;border:1px solid rgba(99,179,237,0.1)'>
                <p style='color:#718096;font-size:0.75rem;margin:0 0 6px'>📍
Location</p>
                <p style='color:#e2e8f0;font-size:0.9rem;margin:0'><b>{city}</b> ·
{season} · {month}/{year}</p>
            </div>
            """, unsafe_allow_html=True)

        # ── Pollutant contribution chart ──
```

```
        st.markdown("<p class='section-header'>🔬 Pollutant Contribution to
AQI</p>", unsafe_allow_html=True)
        weights = {'PM2.5': pm2_5*0.30, 'RSPM': rspm*0.25, 'NO₂': no2*0.20, 'SPM':
spm*0.15, 'SO₂': so2*0.10}
        fig_contrib = px.pie(
            names=list(weights.keys()), values=list(weights.values()),
            color_discrete_sequence=['#e74c3c','#3498db','#2ecc71','#f39c12','#9b59
b6'],
            hole=0.45
        )
        fig_contrib.update_traces(textposition='inside', textinfo='percent+label',
                                  hovertemplate="<b>%{label}</b><br>Weighted:
%{value:.2f}<extra></extra>")
        fig_contrib.update_layout(
            paper_bgcolor='rgba(0,0,0,0)', font_color='#e2e8f0',
            height=320, margin=dict(l=0,r=0,t=10,b=0),
            showlegend=True, legend=dict(font=dict(color='#a0aec0'))
        )
        st.plotly_chart(fig_contrib, use_container_width=True)

    else:
        # Default state
        st.markdown("""
        <div style='text-align:center;padding:60px 20px;'>
            <p style='font-size:4rem;margin:0'>🌫️</p>
            <p style='font-size:1.4rem;color:#e2e8f0;font-weight:600;margin:16px 0
8px'>Ready to Predict</p>
            <p style='color:#718096;font-size:1rem'>Adjust the sliders in the
sidebar and click <b style='color:#90cdf4'>Predict AQI Category</b></p>
        </div>
        """, unsafe_allow_html=True)

        # AQI scale reference
        st.markdown("<p class='section-header'>📋 CPCB AQI Scale Reference</p>",
unsafe_allow_html=True)
        cols_ref = st.columns(6)
        for col, (cat, info) in zip(cols_ref, AQI_CATEGORIES.items()):
            with col:
                lo, hi = info['range']
                st.markdown(f"""
                <div style='background:{info["bg"]};border:1px solid
{info["border"]};border-radius:12px;padding:14px;text-align:center'>
                    <p style='font-size:1.8rem;margin:0'>{info["emoji"]}</p>
                    <p style='color:{info["color"]};font-weight:600;font-
size:0.85rem;margin:6px 0'>{cat}</p>
```

```
                    <p style='color:#718096;font-size:0.75rem;margin:0'>{lo}-
{hi}</p>
                </div>
                """, unsafe_allow_html=True)


# ═══════════════════════════════════════════════════════════════════
#  TAB 2 — ANALYTICS
# ═══════════════════════════════════════════════════════════════════
with tab2:
    st.markdown("<p class='section-header'>📊 Exploratory Analytics Dashboard</p>",
unsafe_allow_html=True)

    # Generate synthetic dataset for analytics
    np.random.seed(42)
    n = 1500
    cities_list =
['Delhi','Mumbai','Chennai','Kolkata','Bengaluru','Hyderabad','Ahmedabad','Pune']
    seasons_list = ['Winter','Spring','Summer','Monsoon']
    city_arr = np.random.choice(cities_list, n)
    season_arr = np.random.choice(seasons_list, n)

    df_viz = pd.DataFrame({
        'city': city_arr,
        'season': season_arr,
        'so2':  np.random.exponential(15, n).round(1),
        'no2':  np.random.exponential(30, n).round(1),
        'rspm': np.random.exponential(80, n).round(1),
        'pm2_5':np.random.exponential(60, n).round(1),
    })
    df_viz['aqi'] = (0.3*df_viz['pm2_5'] + 0.25*df_viz['rspm'] + 0.2*df_viz['no2']
+
                     0.1*df_viz['so2'] + np.random.normal(0, 10,
n)).clip(lower=0).round(1)
    df_viz['category'] = df_viz['aqi'].apply(categorize_aqi)
    # City index for pollution
    city_noise =
{'Delhi':40,'Mumbai':20,'Chennai':10,'Kolkata':30,'Bengaluru':5,'Hyderabad':8,'Ahme
dabad':25,'Pune':12}
    df_viz['aqi'] = (df_viz['aqi'] +
df_viz['city'].map(city_noise)).clip(lower=0).round(1)
    df_viz['category'] = df_viz['aqi'].apply(categorize_aqi)

    col_l, col_r = st.columns(2)
```

```
    with col_l:
        # City AQI comparison
        city_avg =
df_viz.groupby('city')['aqi'].mean().sort_values(ascending=True).reset_index()
        fig_city = px.bar(city_avg, x='aqi', y='city', orientation='h',
                          color='aqi', color_continuous_scale='RdYlGn_r',
                          title='Average AQI by City',
                          labels={'aqi':'Average AQI','city':'City'},
                          text='aqi')
        fig_city.update_traces(texttemplate='%{text:.0f}', textposition='outside')
        fig_city.update_layout(
            paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)',
            font_color='#e2e8f0', height=350, margin=dict(l=0,r=10,t=40,b=0),
            coloraxis_showscale=False,
            title_font=dict(color='#e2e8f0', size=14)
        )
        st.plotly_chart(fig_city, use_container_width=True)

    with col_r:
        # AQI by season violin
        fig_season = px.violin(df_viz, x='season', y='aqi', color='season',
                               box=True, points=False,
                               title='AQI Distribution by Season',
                               color_discrete_sequence=['#3182ce','#38a169','#e53e3
e','#805ad5'],
                               category_orders={'season': seasons_list})
        fig_season.update_layout(
            paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)',
            font_color='#e2e8f0', height=350, showlegend=False,
            margin=dict(l=0,r=0,t=40,b=0),
            title_font=dict(color='#e2e8f0', size=14)
        )
        st.plotly_chart(fig_season, use_container_width=True)

    col_l2, col_r2 = st.columns(2)
    with col_l2:
        # PM2.5 vs AQI scatter
        fig_scatter = px.scatter(df_viz.sample(600), x='pm2_5', y='aqi',
                                 color='category',
                                 color_discrete_map={k: v['color'] for k,v in
AQI_CATEGORIES.items()},
                                 opacity=0.6, title='PM2.5 vs AQI',
                                 trendline='ols')
        fig_scatter.update_layout(
            paper_bgcolor='rgba(0,0,0,0)', plot_bgcolor='rgba(0,0,0,0)',
```

```
            font_color='#e2e8f0', height=350, margin=dict(l=0,r=0,t=40,b=0),
            title_font=dict(color='#e2e8f0', size=14),
            legend=dict(font=dict(color='#a0aec0'))
        )
        st.plotly_chart(fig_scatter, use_container_width=True)

    with col_r2:
        # Category donut
        cat_counts = df_viz['category'].value_counts()
        fig_donut = go.Figure(go.Pie(
            labels=cat_counts.index,
            values=cat_counts.values,
            hole=0.5,
            marker_colors=[AQI_CATEGORIES.get(c, {}).get('color', '#888') for c in
cat_counts.index],
            textinfo='percent+label'
        ))
        fig_donut.update_layout(
            title='AQI Category Share',
            paper_bgcolor='rgba(0,0,0,0)', font_color='#e2e8f0',
            height=350, margin=dict(l=0,r=0,t=40,b=0),
            title_font=dict(color='#e2e8f0', size=14),
            showlegend=False
        )
        st.plotly_chart(fig_donut, use_container_width=True)

    # Correlation heatmap
    st.markdown("<p class='section-header'>🔗 Pollutant Correlation Matrix</p>",
unsafe_allow_html=True)
    corr = df_viz[['so2','no2','rspm','pm2_5','aqi']].corr()
    fig_heat = px.imshow(corr, text_auto='.2f', color_continuous_scale='RdBu_r',
                         zmin=-1, zmax=1, aspect='auto',
                         title='Correlation Heatmap')
    fig_heat.update_layout(
        paper_bgcolor='rgba(0,0,0,0)', font_color='#e2e8f0',
        height=350, margin=dict(l=0,r=0,t=40,b=0),
        title_font=dict(color='#e2e8f0', size=14)
    )
    st.plotly_chart(fig_heat, use_container_width=True)


# ═══════════════════════════════════════════════════════════════════
#  TAB 3 — AQI MAP
# ═══════════════════════════════════════════════════════════════════
with tab3:
```

```
    st.markdown("<p class='section-header'>🗺️ India City AQI Map</p>",
unsafe_allow_html=True)

    city_coords = {
        'Delhi':     (28.6139, 77.2090, 185),
        'Mumbai':    (19.0760, 72.8777, 120),
        'Chennai':   (13.0827, 80.2707,  95),
        'Kolkata':   (22.5726, 88.3639, 155),
        'Bengaluru': (12.9716, 77.5946,  70),
        'Hyderabad': (17.3850, 78.4867,  88),
        'Ahmedabad': (23.0225, 72.5714, 140),
        'Pune':      (18.5204, 73.8567, 105),
    }

    map_df = pd.DataFrame([
        {'City': c, 'Lat': lat, 'Lon': lon, 'AQI': aqi,
         'Category': categorize_aqi(aqi), 'Color': get_aqi_color(aqi)}
        for c, (lat, lon, aqi) in city_coords.items()
    ])

    fig_map = px.scatter_mapbox(
        map_df, lat='Lat', lon='Lon', hover_name='City',
        hover_data={'AQI': True, 'Category': True, 'Lat': False, 'Lon': False},
        color='AQI', size='AQI', size_max=40,
        color_continuous_scale='RdYlGn_r',
        zoom=4.5, center={'lat': 22, 'lon': 80},
        height=520,
        title='City-wise AQI Overview (Sample Data)'
    )
    fig_map.update_layout(
        mapbox_style='carto-darkmatter',
        paper_bgcolor='rgba(0,0,0,0)',
        font_color='#e2e8f0',
        margin=dict(l=0,r=0,t=40,b=0),
        title_font=dict(color='#e2e8f0', size=14)
    )
    st.plotly_chart(fig_map, use_container_width=True)

    # City table
    st.markdown("<p class='section-header'>📋 City AQI Summary</p>",
unsafe_allow_html=True)
    for _, row in map_df.sort_values('AQI', ascending=False).iterrows():
        cinfo = AQI_CATEGORIES[row['Category']]
        bar_width = min(int(row['AQI'] / 5), 100)
        st.markdown(f"""
```

```
        <div style='display:flex;align-items:center;gap:12px;margin-bottom:8px;
                    background:rgba(255,255,255,0.02);border-
radius:10px;padding:10px 14px'>
            <span style='width:120px;color:#e2e8f0;font-
weight:500'>{row["City"]}</span>
            <div style='flex:1;background:rgba(255,255,255,0.05);border-
radius:6px;height:10px'>
                <div
style='width:{bar_width}%;background:{cinfo["color"]};height:10px;border-
radius:6px'></div>
            </div>
            <span style='color:{cinfo["color"]};font-weight:600;width:50px;text-
align:right'>{int(row["AQI"])}</span>
            <span style='color:#718096;font-
size:0.8rem;width:90px'>{row["Category"]}</span>
        </div>
        """, unsafe_allow_html=True)


# ═══════════════════════════════════════════════════════════════════════
#  TAB 4 — ABOUT
# ═══════════════════════════════════════════════════════════════════════
with tab4:
    st.markdown("<p class='section-header'>ℹ️ About This Project</p>",
unsafe_allow_html=True)

    col_a, col_b = st.columns(2)
    with col_a:
        st.markdown("""
        <div style='background:rgba(49,130,206,0.08);border:1px solid
rgba(49,130,206,0.2);border-radius:16px;padding:24px'>
            <h3 style='color:#90cdf4;margin-top:0'>🎯 Project Objective</h3>
            <p style='color:#a0aec0;line-height:1.8'>
                To predict the Air Quality Index (AQI) category of Indian cities
using a
                <b style='color:#e2e8f0'>Logistic Regression</b> model trained on
historical pollutant
                and weather data, enabling early warnings for public health
protection.
            </p>
            <h3 style='color:#90cdf4'>🧮 Algorithm</h3>
            <p style='color:#a0aec0'>Multinomial Logistic Regression with:</p>
            <ul style='color:#a0aec0;line-height:2'>
                <li>LBFGS / SAGA solver</li>
                <li>L2 Regularization (tuned via GridSearchCV)</li>
```

```
                <li>Class-weight balancing</li>
                <li>5-fold Stratified Cross-Validation</li>
            </ul>
        </div>
        """, unsafe_allow_html=True)

    with col_b:
        st.markdown("""
        <div style='background:rgba(56,161,105,0.08);border:1px solid
rgba(56,161,105,0.2);border-radius:16px;padding:24px'>
            <h3 style='color:#68d391;margin-top:0'>📊 Dataset</h3>
            <ul style='color:#a0aec0;line-height:2'>
                <li><b style='color:#e2e8f0'>Source:</b> Kaggle — India Air Quality
Data</li>
                <li><b style='color:#e2e8f0'>Records:</b> ~2,000+ city-day
observations</li>
                <li><b style='color:#e2e8f0'>Features:</b> SO₂, NO₂, RSPM, SPM,
PM2.5, temp, humidity</li>
                <li><b style='color:#e2e8f0'>Target:</b> AQI Category (6 classes,
CPCB)</li>
            </ul>
            <h3 style='color:#68d391'>⏳ ML Pipeline</h3>
            <ul style='color:#a0aec0;line-height:2'>
                <li>Data Cleaning + Feature Engineering</li>
                <li>StandardScaler normalization</li>
                <li>GridSearchCV hyperparameter tuning</li>
                <li>ROC-AUC, F1, Precision, Recall evaluation</li>
            </ul>
        </div>
        """, unsafe_allow_html=True)

    st.markdown("""
    <div style='background:rgba(159,122,234,0.08);border:1px solid
rgba(159,122,234,0.2);border-radius:16px;padding:24px;margin-top:16px'>
        <h3 style='color:#b794f4;margin-top:0'>✨ Unique Features of This App</h3>
        <div style='display:grid;grid-template-columns:1fr 1fr 1fr;gap:12px'>
            <div style='background:rgba(159,122,234,0.08);border-
radius:10px;padding:14px'>
                <p style='color:#b794f4;font-weight:600;margin:0 0 6px'>🎯 Real-
Time Gauge</p>
                <p style='color:#a0aec0;font-size:0.85rem;margin:0'>Interactive AQI
gauge with live pollutant contribution breakdown</p>
            </div>
            <div style='background:rgba(159,122,234,0.08);border-
radius:10px;padding:14px'>
```

```
                <p style='color:#b794f4;font-weight:600;margin:0 0 6px'>🗺️ India
AQI Map</p>
                <p style='color:#a0aec0;font-size:0.85rem;margin:0'>Interactive
Mapbox city-level pollution visualization</p>
            </div>
            <div style='background:rgba(159,122,234,0.08);border-
radius:10px;padding:14px'>
                <p style='color:#b794f4;font-weight:600;margin:0 0 6px'>🩺 Health
Advice</p>
                <p style='color:#a0aec0;font-size:0.85rem;margin:0'>CPCB-aligned
health advisories per predicted category</p>
            </div>
        </div>
    </div>
    """, unsafe_allow_html=True)

    st.markdown("""
    <div style='text-align:center;margin-top:32px;padding:20px;color:#4a5568;font-
size:0.8rem'>
        Built with 💜 using Python · Scikit-learn · Streamlit · Plotly<br>
        <b>AQI Guardian</b> · International ML Competition Project · 2025
    </div>
    """, unsafe_allow_html=True)
```